

Programação Funcional em Haskell

Composição monádica

REALIZAR APÓS posição 44: 11.9 - Programação Funcional em Haskell: Funções de Alta Ordem para Monads

Posições na playlist 36 a 44 | 120 minutos | 100 pontos

Curso principal: Programação Funcional em Haskell - Curso Completo - UFABC |
<https://www.youtube.com/watch?v=eTisiy5FB7k&list=PLYItvall0TqJ25sVTLcMhxsE0Hci58mpQ&index=1>

Conteúdo verificável: Monad; listas; Either; Reader; Writer; State; IO; combinadores

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Maybe e Either

(a) Encadeie divisão segura por 2 e depois por 0 em Maybe usando($>>=$). (b) Faça versão Either String com mensagem. (c) Explique diferenças entre ausência e erro informado e indique lei de identidade do bind.

Desenvolvimento / justificativa / esquema:

Monad de listas

(a) Avalie $[1, 2] >>= (\backslash x \rightarrow [x, -x])$. (b) Escreva compreensão equivalente. (c) Explique como encadeamento representa escolhas e difere de produto cartesiano applicative em uma construção concreta.

Desenvolvimento / justificativa / esquema:

Reader, Writer e State

Associe cada necessidade a Reader, Writer ou State: configuração compartilhada, registro de eventos, contador atualizado. (a) Dê uma computação exemplo para cada. (b) Explique entrada/saída abstrata. (c) Diferencie estado explícito de mutação global descontrolada.

Desenvolvimento / justificativa / esquema:

IO e combinadores

(a) Explique o tipo de ação que lê texto e imprime seu comprimento. (b) Compare mapM e map puro para ações de IO. (c) Por que não se deve tratar IO String como uma String já obtida?

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Maybe e Either (15 pontos)

Divisão segura $x \div d = \text{if } d \neq 0 \text{ then } \text{Just}(x/d) \text{ else } \text{Nothing}$; bind interrompe ao Nothing na segunda etapa. Either usa Left 'divisao por zero' e Right resultado; Left é preservado. $\text{return/pure } a \gg= f = f a$ e $m \gg= \text{return } m$ são identidades com tipos/contextos adequados. Either carrega informação de falha; Maybe apenas ausência. Não avaliar a divisão inválida antes de testar denominador.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Monad de listas (25 pontos)

Resultado $[1, -1, 2, -2]$; compreensão $[y \mid x \in [-1, 2], y \in \{-x, x\}]$. Bind concatena resultados que podem depender de cada x . Applicative de listas aplica cada função a cada argumento, combinando escolhas independentes; Monad permite construir próximas alternativas em função do valor anterior. A ordem dos resultados depende da ordem de enumeração da instância.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Reader, Writer e State (25 pontos)

Reader recebe ambiente e consulta sem modificá-lo; Writer devolve valor e log combinado por monoid; State representa transformação estado $\rightarrow$ (valor, novo estado). Exemplos: calcular preço com taxa da configuração; anotar etapas; produzir identificador e incrementar contador. Efeitos são organizados nas interfaces e composição, não variáveis globais implícitas. Ordem e combinação do log/estado devem ser especificadas.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - IO e combinadores (35 pontos)

Exemplo de ação: $\text{getLine} \gg= (\lambda s \rightarrow \text{print}(\text{length } s))$, $\text{tipIO}()$. getLine é ação IO String; bind disponibiliza resultado dentro da composição de ações. map sobre ações constrói estruturas; mapM organiza a execução e coleta resultados no contexto. IO String representa computação com efeitos, não texto puro disponível sem executar no contexto apropriado. A ordem de efeitos é parte do programa.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.